

Week 2 Goal

By the end of Week 2, you should have:

- Real job data entering the system automatically
- Scheduled workflows running without manual execution
- Job APIs or RSS feeds integrated
- AI scoring happening automatically
- Jobs being stored in Supabase continuously
- Discord notifications for new high-match jobs
- Basic filtering/ranking logic
- The first “useful” version of the system

If you complete ONLY Week 1 + Week 2:

you already have a genuinely impressive AI-powered automation platform.

Week 2 Architecture

Job APIs / RSS Feeds



n8n Scheduled Workflow



Data Cleanup / Formatting



OpenAI Job Scoring



Supabase Storage



Discord Notifications

PRIMARY OBJECTIVE OF WEEK 2

Replace:

fake/sample jobs

with:

real continuously updating jobs

This is the transition from:

- demo project
 - to:
 - real useful system
-

IMPORTANT RULES FOR WEEK 2

Rule 1

Do NOT touch LinkedIn automation yet.

Rule 2

Do NOT build frontend/dashboard yet.

Rule 3

Do NOT build multi-agent systems yet.

Rule 4

Do NOT optimize architecture yet.

Rule 5

Focus ONLY on:

- real data ingestion
- workflow reliability
- automation scheduling
- filtering/ranking

PHASE 1 — Real Job Sources

Step 1 — Choose Job Sources

Use:

- Greenhouse job boards
- Lever job boards
- RSS feeds
- company career APIs
- public JSON feeds

Avoid:

- LinkedIn scraping
 - browser automation
 - CAPTCHA-heavy systems
-

Recommended Early Targets

Greenhouse Companies

Many startups use:

boards.greenhouse.io - basically just use

“https://boards-api.greenhouse.io/v1/boards/{company_name}/jobs” to find if the company has jobs posted

Example:

- OpenAI
- Anduril
- Stripe
- Ramp

Lever Companies

Many tech companies use:

`jobs.lever.co`

These are easy to query.

Step 2 — Create “Target Companies” List

Create:

`docs/target_companies.md`

Include:

- Nvidia
- Lockheed Martin
- Anduril
- Microsoft
- Delta
- NCR
- Google
- SAIC
- KBR
- defense contractors
- Atlanta companies

This becomes your filtering foundation.

PHASE 2 — Scheduled Automation

Step 3 — Add Schedule Trigger

Replace:

Manual Trigger

with:

Schedule Trigger

Suggested:

Every 6 hours

This makes the system autonomous.

Step 4 — Add HTTP Request Node

Workflow:

Schedule Trigger



HTTP Request



Parse Jobs



AI Scoring



Supabase



Discord

The HTTP node will:

- fetch live job data
 - return JSON job listings
-

Step 5 — Parse Real Job Data

You now need to:

- extract title
- company
- location
- description
- URL

This is where you start learning:

- real JSON parsing
 - data transformation
-

PHASE 3 — AI Job Filtering

Step 6 — Improve Prompt

Instead of:

simple scoring

now include:

- salary potential
 - AI relevance
 - defense relevance
 - new grad friendliness
 - embedded systems relevance
 - location preference
-

Example Prompt

You are evaluating jobs for a computer engineering student interested in:

- AI
- embedded systems
- defense technology
- high-income career growth
- Atlanta/Huntsville opportunities

Score from 1-100.

Prioritize:

- strong technical growth
- AI exposure
- defense/autonomy systems
- compensation potential
- reputable companies

Return ONLY JSON:

```
{  
  "score": number,  
  "reason": "short explanation"  
}
```

Step 7 — Add Filtering Logic

Add an IF node.

Example:

Only continue if score > 75

This prevents spam.

Now Discord only receives:

- high-quality opportunities

This makes the system feel much smarter.

PHASE 4 — Improve Database Design

Step 8 — Expand Jobs Table

Add fields:

```
alter table jobs  
add column url text;
```

```
alter table jobs  
add column source text;
```

```
alter table jobs  
add column ai_relevance integer;
```

```
alter table jobs  
add column processed_at timestamp default now();
```

This prepares for:

- deduplication
 - tracking
 - analytics later
-

Step 9 — Prevent Duplicate Jobs

This is IMPORTANT.

You do NOT want:

- repeated Discord spam
- repeated DB inserts

Simple strategy:

- check if URL already exists before insert

This introduces:

- database querying
- conditional logic

Very important backend concept.

PHASE 5 — Better Discord Notifications

Step 10 — Improve Discord Formatting

Make notifications cleaner:

New High-Match Job

Company: Nvidia

Role: Embedded AI Engineer

Location: Santa Clara, CA

Match Score: 89

Reason:

Excellent AI systems alignment with strong compensation potential.

Apply:

<job URL>

This makes the system feel professional.

PHASE 6 — Organize Workflow Structure

Step 11 — Export Workflows

Save:

workflows/

Examples:

job_ingestion.json

job_scoring.json

discord_alerts.json

Step 12 — Add Better Documentation

Document:

- APIs used
- workflow architecture
- database schema
- scoring methodology

This becomes resume/interview material later.

Stretch Goals (ONLY if ahead)

Optional:

- Telegram notifications
 - multiple job sources
 - company scoring
 - daily digest workflow
 - AI-generated summaries
 - recruiter/company tagging
-

IMPORTANT LESSONS OF WEEK 2

By the end of Week 2 you will understand:

- scheduled automation

- API ingestion
- JSON transformation
- filtering logic
- database querying
- autonomous workflows
- event-driven systems
- production workflow debugging

This is where the project transitions from:

cool demo

to:

actually useful system

What Week 2 Teaches You

Most importantly:

you begin learning that real AI systems are mostly:

- infrastructure
- data pipelines
- orchestration
- integrations
- automation

NOT just prompting.

That realization is extremely valuable.

What You Will Have After Week 2

A system that:

- continuously monitors real jobs
- evaluates them using AI
- stores them automatically

- filters high-quality matches
- sends intelligent alerts

At this point, the project already becomes:

- resume-worthy
- demo-worthy
- interview-worthy
- genuinely useful for your own career search